1. A Hamiltonian cycle in a graph is a cycle that visits every vertex exactly once. A Hamiltonian path in a graph is a path that visits every vertex exactly once, but it need not be a cycle (the last vertex in the path may not be adjacent to the first vertex in the path.) Consider the Directed Hamiltonian Cycle problem, which checks whether a Hamiltonian cycle exists in a directed graph, the Undirected Hamiltonian Cycle problem, which checks whether a Hamiltonian cycle exists in an undirected graph, and the Undirected Hamiltonian Path problem, which checks whether a Hamiltonian path exists in an undirected graph.

a. Give a polynomial time reduction from the *directed* Hamiltonian cycle problem to the *undirected* Hamiltonian cycle problem.

Solution: For any arbitrary directed graph $G_d := \{V_d, E_d\}$, construct the following undirected graph $G_u := \{V_u, E_u\}$:

- $V_u := \{v_{in}, v_{mid}, v_{out} \mid v \in V_d\}$. For each of the vertices in the directed graph, we split them into a triplet of in, mid, and out.
- $E_u := \left\{(u_{out}, v_{in}) \middle| (u,v) \in E_d\right\} \cup \left\{(v_{in}, v_{mid}), (v_{mid}, v_{out}) \middle| v \in V_d\right\}$. For each of the triplets that comes from the same vertex, we connect them in the order of in-mid-out. The directed edges in the V_d become the undirected ones that connect out and in between corresponding triplets.

Notice that $|V_u| = 3|V_d|$ and $|E_u| = |E_d| + 2|V_d|$, so this reduction is linear.

$\Rightarrow$: Suppose that in G_d there exists a Hamiltonian cycle $C_d := \left(c_1, c_2, \ldots, c_{|V_d|}\right)$, where $c_i \in V_d$. Then in G_u there should also exist

$$C_u := \left(c_{1in}, c_{1mid}, c_{1out}, c_{2in}, c_{2mid}, c_{2out}, \ldots, c_{|V_d|_{in}}, c_{|V_d|_{mid}}, c_{|V_d|_{out}}\right),$$

which is a Hamiltonian cycle in G_u.

$\Leftarrow$: Suppose that in G_u there exists a Hamiltonian cycle C'_u. By definition, within each of the triplets there should only be a path of order in-mid-out, and between two triplets there should only be an edge of out-in. Thus, C'_u should always be of the following form

$$C'_u := \left(c'_{1in}, c'_{1mid}, c'_{1out}, c'_{2in}, c'_{2mid}, c'_{2out}, \ldots, c'_{|V_d|_{in}}, c'_{|V_d|_{mid}}, c'_{|V_d|_{out}}\right),$$

which corresponds to a Hamiltonian cycle $C'_d := \left(c'_1, c'_2, \ldots, c'_{|V_d|}\right)$ in G_d.

■

b. Give a polynomial time reduction from the *undirected* Hamiltonian Cycle to *directed* Hamiltonian cycle.

Solution: This reduction is simpler than the previous one. Given an instance G of undirected Hamiltonian cycle, Let G' be the directed graph with the same vertices as G and containing edges $u \to v$ and $v \to u$ for every edge $uv \in G$.

($\Rightarrow$) If C is a cycle in G, then C is also a cycle in the directed graph G'. For every $u \to v \in G'$, the edge uv is in G.

($\Leftarrow$) If C is a cycle in G', then C is also a cycle in the original graph G. For every $uv \in G$, the edge $u \to v \in G'$ by the construction.

■

c. Give a polynomial-time reduction from an undirected Hamiltonian *path* to an undirected Hamiltonian *cycle*.

Solution: Let the input to this problem be an undirected graph G. The goal is to produce G' such that G has a Hamiltonian path if G' has a Hamiltonian cycle.

This can be done by adding a vertex v with edges to every vertex in the original graph G, this will be G'.

$\Rightarrow$: If there exists a Hamiltonian path P in G, starting with vertex s and ending with vertex t. Then $[v, s, P, t, v]$ is a Hamiltonian cycle in G'.

$\Leftarrow$: In the other case, if C is the Hamiltonian cycle in G', then removing v from C will return a Hamiltonian path in G.

■

2. For each of the following problems, pick true or false and explain why.

   (a) **True/False:** If L is an NP-complete language and $L \in P$, then $P = NP$.

   Solution: True. If L is an NP-complete language and $L \in P$, then $P = NP$. This is because NP-complete problems are the hardest problems in NP, and if any NP-complete problem is in P, then all problems in NP are in P. ■

   (b) **True/False:** There exists a polynomial-time reduction from every problem in NP to every problem in P.

   Solution: False. There does not necessarily exist a polynomial-time reduction from every problem in NP to every problem in P. A reduction must map a problem in NP to another problem in NP or a problem in P to another problem in P. ■

   (c) **True/False:** If a problem is both NP-hard and co-NP-hard, then it must be in NP.

   Solution: False. If a problem is both NP-hard and co-NP-hard, it does not necessarily imply it is in NP. Being NP-hard means every problem in NP can be reduced to it in polynomial time, and co-NP-hard means every problem in co-NP can be reduced to it in polynomial time. ■

   (d) **True/False:** If there is a polynomial-time reduction from problem A to problem B and B is in NP, then A must also be in NP.

   Solution: True. If there is a polynomial-time reduction from problem A to problem B and B is in NP, then A must also be in NP. This is because the reduction allows us to transform A into B, and since B is in NP, the transformed instance can be verified in polynomial time, implying that A is also in NP. ■

   (e) **True/False:** If a problem is solvable in polynomial space, then it is also solvable in polynomial time.

   Solution: False. If a problem is solvable in polynomial space, it does not necessarily mean it is solvable in polynomial time. Polynomial space includes problems that might take exponential time but use polynomial memory, such as PSPACE-complete problems. ■

3. You are given a set U of n elements, a collection of m subsets $S_1, S_2, \ldots, S_m$, where each $S_i \subseteq U$, and a positive integer k. The Set Packing problem asks whether there exists a collection of at least k subsets such that no two selected subsets share any elements.

   Prove that Set Packing is NP-hard. Give a polynomial-time reduction from the Independent Set problem to Set Packing and explain why your reduction is correct.

Solution: We reduce from the Independent Set problem, which is known to be NP-complete. Given an instance of Independent Set consisting of an undirected, unweighted graph $G = (V, E)$ and a positive integer k, we construct an instance of Set Packing as follows.

For each edge $(u, v) \in E$, create a unique element $x_{u,v}$, and let the universe U be the set of all such elements. Then, for every vertex $v \in V$, define a subset $S_v \subseteq U$ such that $S_v = \{x_{u,v} : (u, v) \in E\} \cup \{x_{v,u} : (v, u) \in E\}$. That is, S_v contains all elements corresponding to edges incident to vertex v. Set the parameter k in the Set Packing instance equal to the k in the Independent Set instance.

We now show correctness in both directions:

(1) Suppose there exists an independent set of size at least k in the original graph. Let $S \subseteq V$ be such a set. For each vertex $v \in S$, include the subset S_v in the Set Packing solution. Since no two vertices in S share an edge, the subsets $\{S_v : v \in S\}$ are pairwise disjoint. Therefore, this forms a valid packing of at least k subsets.

(2) Suppose there exists a packing of at least k pairwise disjoint subsets in the Set Packing instance. Let $C \subseteq \{S_v\}$ be such a collection. If two subsets $S_u, S_v \in C$ had a common element $x_{u,v}$, then there would be an edge $(u, v) \in E$, which contradicts the assumption that S_u and S_v are disjoint. Hence, the corresponding vertices form an independent set of size at least k in the original graph.

The reduction takes $O(n+m)$ time, where $n = |V|$ and $m = |E|$, since we construct one element per edge and one subset per vertex. Thus, the reduction is polynomial-time. ■